

Extended Abstract

Motivation As large language model performance continues to improve, further improvements in fine-tuning require increasingly varied quality training data to provide sufficient exploration to make improvements. As LLM training data approaches full Internet scale, this high-quality data has become difficult to obtain. Thus, alternative methods to explore and generate novel responses to encourage additional learning are extremely attractive for continued fine-tuning of large language models.

Method We propose multi-agent self-play to generate diverse chains of thought to aid both in exploration during training and methodical reasoning during inference. We implement a variant of Subramaniam et al. (2025), where we construct a multiagent setup in which multiple fine-tuned language models participate in debate, digesting all responses from previous rounds to generate a better response. We intentionally construct training dataloaders to exclusively train each model on their own responses (either good generations or correct debate performance), to encourage diversity in responses. Thus, by fine-tuning multiple diverse agents, the multiagent system can explore more of the search space and reason over them to yield increased performance.

Implementation To evaluate this proposed method, we implement and apply a variety of fine-tuning methods onto the Qwen2.5-0.5B baseline pretrained model. We then evaluate the performance of these finetuned models on Ultrafeedback and Countdown tasks. In particular, we use supervised fine-tuning (SFT) to specialize models for Countdown and Ultrafeedback, then finetune them with Direct Preference Optimization (DPO) for Ultrafeedback preference data and REINFORCE Leave-One-Out (RLOO) for countdown. Finally, we apply the multi-agent framework to finetune for the Countdown task.

Results For the Countdown task, upon evaluation we find that SFT strictly improves on baseline, as does RLOO over SFT. RLOO is able to achieve average score of 0.4 on the Countdown task. The multi-agent self-play framework demonstrates interesting behavior, in which fully-incorrect (ie incorrect formatting) answers are essentially eradicated, while the rate of correct answers decreases when compared to RLOO, ultimately yielding a final score of 0.345. We also find that additional rounds of debate (especially after 4 rounds of debate) yields diminishing returns on performance improvement.

For the Ultrafeedback task, we found that the SFT agent significantly outperforms the baseline, and the DPO agent slightly outperforms the SFT agent, as expected.

Discussion From these results, we can see some interesting patterns, although to this point in training RLOO still appears to be the best option. In particular, we highlight the ability of self-play to clean up poorly-formatted examples almost entirely, likely stemming from the multi-agent formulation’s multiple chances to come up with an answer, both on initial generation and in subsequent rounds of debate. Overall, we found that managing GPU memory during simultaneous training of multiple models to be the biggest bottleneck on improving performance. Additionally, the variable size of datasets, and effectively fitting different loss functions on each model simultaneously made training finicky and hard to tune. Through additional computational resources, a more parallelized training pipeline, and experimenting with alternative prompt formulation, different baseline models, we are confident that this method could improve beyond its RLOO baseline.

Conclusion Overall, we find that multi-agent self-play and debate for diverse reasoning yields interesting behavior when compared with RLOO and other more traditional language model fine-tuning methods. Although our implementation did not strictly improve on the RLOO baseline, we were able to almost fully eradicate ill-formed responses, pointing to a promising ability for multi-agent formulations to catch and correct blatant errors in responses from each other, by providing the agents multiple opportunities to generate a coherent answer. As language models training pipelines become increasingly starved for new and diverse datasets, multi-agent methods like the one presented here offer a pathway for continuous improvement, towards the next generation of language models.

Debate for Multiagent Large Language Model Tuning

Andrew Wang

Department of Mechanical Engineering
Stanford University
aw1605@stanford.edu

Abstract

As large language model performance continues to improve, further improvements in fine-tuning require increasingly varied quality training data to provide sufficient exploration to make improvements. We propose multi-agent self-play to generate diverse chains of thought to aid both in exploration during training and methodical reasoning during inference. We implement a variant of Subramaniam et al. (2025), where we construct a multiagent setup in which multiple fine-tuned language models participate in debate, digesting all responses from previous rounds to generate a better response. We benchmark this method against several other methods from the Qwen2.5-0.5B baseline model, including SFT, RLOO, and DPO, and show that the method produces results which eradicate ill-formed responses, while suffering a decrease in accuracy compared to the RLOO baseline. Despite these mixed results, multi-agent methods like this one offer a promising alternative to provide new data to build the next generation of large language models.

1 Introduction

Recent advancements in large language models are fundamentally based on a foundation of immense Internet-scale data the models are trained on Kaplan et al. (2020). Thus, a challenge for continued improvement of LLMs is the collection and curation of high-quality training data, especially as that training data approaches full Internet scale.

One promising approach to address this fundamental constraint is fine-tuning using self-play, famously applied to AlphaGo and similar game-playing agents to explore and generate more diverse scenarios on which reinforcement learning algorithms can continue to fine-tune Silver et al. (2017). In this project, we seek to apply concepts of self-play to generate new and diverse scenarios during training to iteratively fine-tune the language model for improved performance. Specifically, through careful curation of data and design of the multi-agent structure and interaction, we aim to encourage fine-tuning of **diverse** agents, which can increase diversity of training, and debate to generate higher-quality responses at inference.

In this project we seek to evaluate the potential performance gains that multi-agent language models may unlock. In particular, we wish to answer the following questions:

1. How does multi-agent fine-tuning improve language model performance?
2. How does training for diversity in reasoning affect the overall performance of the system?

2 Related Work

Reinforcement learning has emerged as the predominant method for fine-tuning large language models, with near-ubiquitous use among all leading language models. Wang et al. (2024). Many methods exist to pose LLM fine-tuning as a reinforcement learning problem, the most popular of

which include preference optimization and online methods. Preference optimization uses datasets containing expert-labeled preferences between two proposed responses, and fit a reward model for the agent to fine-tune its behavior towards optimizing. For this project, we apply Direct Preference Optimization (DPO) Rafailov et al. (2023), which uses a clever parameterization to use an *implicit* reward model to guide training on preference data. Online methods are also popular, using online generations for sample prompts and a reward model to provide live feedback for current model performance. For this project, we use REINFORCE Leave-One-Out (RLOO) Ahmadian et al. (2024), which found that a simpler REINFORCE-style algorithm can outperform popular methods like PPO at lower computational cost.

The key drawback these methods share (as with many similar reinforcement learning methods) boils down to data availability. Preference training relies on the existence of curated preference datasets, which may be expensive to collect. Online training requires policy evaluation in-the-loop, which again is expensive in computational time and resources. As new quality data becomes harder to collect, we turn to alternative methods which generate synthetic data to self-improve language models, thereby bypassing the fundamental limit on the amount of available Internet data. By generating new data via multiagent exploration, we aim to encourage diverse scenarios to train more robust LLM performance. For example, Ouyang et al. (2022) fits a learned model to approximate human feedback, and then uses that model to guide training of the LLM with a classic RL algorithm like PPO. Other alternative strategies leverage LLMs to provide feedback, eg by converting natural language to reward functions, and using the LLM to self-reward as part of the training process Wang et al. (2024). Empirically, previous works which have implemented fine-tuning with self-improvement Chen et al. (2024) Yuan et al. (2024), experience quickly diminishing returns as diversity in the LLM’s responses decreases. This motivates the use of a multi-agent approach which maintains diversity in reasoning even as we iteratively fine-tune, ideally improving robustness of the overall system at each iteration Subramaniam et al. (2025).

3 Method

3.1 Supervised Fine-Tuning (SFT)

Following pretraining, supervised fine-tuning is used to align a model to the specific application task. We use a dataset of examples of responses/generations for tasks within the application domain, and learn on per-token pretraining loss to imitate that dataset. This is essentially the cross-entropy loss over the generated tokens.

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{x,y \in D} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t | x, y_{<t}) \quad (1)$$

3.2 Direct Preference Optimization (DPO)

To leverage preference datasets, we use an implicit-reward-model method called Direct Preference Optimization (Rafailov et al. (2023)), which avoids explicitly fitting a reward model, and instead performs direct updates on the policy from the preference data. The objective is equivalent to encouraging the preferred generation and discouraging the dispreferred generation, while not diverging too much from the (baseline) reference model.

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \quad (2)$$

In our implementation, we use a variant proposed in Pal et al. (2024), which aims to address a degenerate condition during training where the probability of generating preferred responses is suppressed, but dispreferred sample is suppressed more, resulting in poor generations while the loss function decreases. The method accomplishes this by introducing an additional penalty for the samples where the preferred response probability is lower for the currently training model than the reference model.

$$\mathcal{L}_{\text{DPOP}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \left(\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) - \lambda \cdot \max \left(0, \log \frac{\pi_{\text{ref}}(y_w|x)}{\pi_\theta(y_w|x)} \right) \right) \right] \quad (3)$$

3.3 REINFORCE Leave-One-Out (RLOO)

The countdown task, which has a closed-form rule-based verifier reward function, is well-suited for traditional policy gradient reinforcement learning methods. Here, we use RLOO, from Ahmadian et al. (2024), which is simply vanilla REINFORCE, but with a clever baseline computed from the average reward from the rest of the k -sized batch (ie baseline for each sample is computed by "leaving out" that sample and calculating the mean for the rest of the batch).

$$\mathcal{L}_{\text{RLOO}}(\theta) = -\frac{1}{k} \sum_{i=1}^k \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi(y_{(i)} | x) \quad \text{for } y_{(1)}, \dots, y_{(k)} \stackrel{\text{i.i.d.}}{\sim} \pi_\theta(\cdot | x) \quad (4)$$

3.4 Multi-Agent Actor-Critic Self-Improvement with Diverse Reasoning Chains

We propose an alternative approach to large language model fine-tuning, which leverages multi-agent debate to improve finetuning performance, based on the methods proposed in Subramaniam et al. (2025).

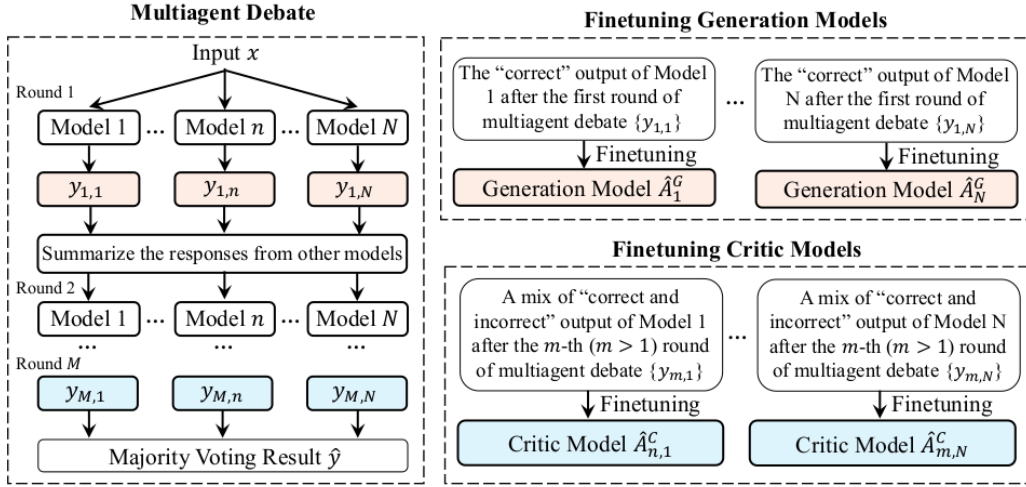


Figure 1: Overview of Multi-Agent Fine-Tuning Subramaniam et al. (2025)

The method relies on a few key components:

1. **Generation Agents:** Agents fine-tuned to generate responses from scratch to the raw prompt.
2. **Critic Agents:** Agents fine-tuned to digest both the original prompt, as well as responses from other agents, to output a response based on that aggregated information.
3. **Multi-Agent Debate** Du et al. (2023): Critics iteratively process responses from each critic in the previous round to synthesize a new response.

During the training process, the training data pipeline is explicitly constructed to feed responses into different models, with the express goal of differentiating the generation/critic agents to promote diverse chains of reasoning. In particular, we store (1) generations which ultimately "won" the majority vote at the end of the debate, and (2) critic responses which either correctly changed their mind (to match the majority opinion) or correctly held their ground (resulting in "convincing" the other models to agree). Intuitively, we are trying to encourage both accurate generations, as well as

debate behavior that leads to convergence to the consensus answer at the end of debate. The details for the method are summarized in detail in Algorithm 1.

Our proposed method extends on Subramaniam et al. (2025) in two key ways. First, instead of maintaining separate generation and critic models, we use a single set of models for both generating and critiquing responses. This effectively halves the memory required for the multiagent system. We hypothesize that, for the specific task at hand (ie Countdown), the reasoning between generation and analysis of solutions is roughly the same, and that we can recover the same level of performance without maintaining separate generation and critic models throughout training. The second change we make is to the training algorithm. Instead of constructing a full dataset on every task before performing any finetuning steps, we use a replay buffer and iteratively add to and sample from a replay buffer to continually finetune the multiagent system as we collect data. Essentially, we apply a similar structure to Soft Actor-Critic in Haarnoja et al. (2018). Once the dataloaders are constructed, we simply perform SFT updates on batches of the desirable outputs for each model.

The method relies on a few key insights. By promoting diversity between multiple agents, finetuning LLMs can avoid plateau in improvement, which is caused by poor exploration of answers stemming from too-similar generations. This provides benefits both at training time (by improving exploration) and at inference time (by considering many options before answering), albeit at the cost of computational resources. The effectiveness of this method also depends on the fact that language models tend to be better at critiquing responses than generating raw outputs from scratch. Additionally, since language models can be easily derailed by poor chain of thought, multiple agents provide a way for multiple chains of reasoning to be explored, then evaluated.

For our implementation, we initialize each individual agent in the multi-agent framework using our RLOO agent. We use $N = 3$ agents and sweep over M rounds of debate up to 8 total rounds (the first round is simply the generation step).

4 Experimental Setup

We benchmark the performance and general applicability of these methods by fine-tuning a large language model on two tasks: Countdown and Ultrafeedback. For both tasks, we start from the pretrained Qwen2.5-0.5B baseline model, a 0.5-billion parameter language model trained for multilingual, long-context support for good performance in tasks like instruction following, reasoning, and producing structured outputs QwenTeam (2024).

In Countdown, the model must reason over a list of 3-4 numbers, and use arithmetic operations to combine them into a target number, demonstrating coherent mathematical reasoning. For this task, we rely on the WarmStart dataset for supervised fine-tuning (adapting to the proper formatting), and the larger Countdown dataset for a large corpus of sample Countdown problems for RL training and evaluation. To evaluate Countdown performance, we use a rule-based verifier to check for correct formatting and mathematical correctness of the final answer Gandhi et al. (2025).

The Ultrafeedback task requires high-quality and well-formatted responses to follow instructions from a diverse set of language prompts. For this task, we train on the Smoltalk dataset for supervised fine-tuning and the larger Ultrafeedback Binarized preference dataset for RL training and evaluation. To evaluate Ultrafeedback performance, we use win rate for head-to-head comparisons of generated prompts between the eval model and a baseline model, using the nvidia/Llama-3.1-Nemotron-70B-Reward model online inference API.

For the multi-agent model, we perform some additional analysis to shed some light on how the multiple models interact with one another. In particular, we are interested in the qualitative behavior and diverse answers through the different rounds of debate. Additionally, we seek to quantify the effect of varying the number of debate rounds on the overall performance of the system.

5 Results

See the appendix for details on training hyperparameters used to generate the following results.

Algorithm 1 Offline Multiagent Finetuning of Language Models

Require: A pretrained LLM A ; A set of language inputs $\mathcal{D}_{\text{task}} = \{x_i\}$; The number of agents N ; The number of debate rounds M ; The number of finetuning iterations L ; The number of training steps per iteration T .

```
1:  $A_1, \dots, A_N \leftarrow A$  ▷ Copy the LLM to build  $N$  generation/critic agents
2: Initialize datasets for finetuning generation models  $\{\mathcal{D}_n^G\}_{n=1}^N$ 
3: Initialize datasets for finetuning critic models  $\{\mathcal{D}_n^C\}_{n=1}^N$ 
4: for  $l = 1$  to  $L$  do ▷ Multiple Iterations of Finetuning
5:   for  $x$  in  $\mathcal{D}_{\text{task}}$  do ▷ Multiagent Debate
6:     for  $m$  in  $M$  do
7:       if  $m = 0$  then
8:          $y_{1,1}, \dots, y_{1,N} \leftarrow A_1(x), \dots, A_N(x)$  ▷ Response of each generation agent
9:       else
10:         $y_{m,1}, \dots, y_{m,N} \leftarrow A_1(x_{m,1}^s), \dots, A_N(x_{m,N}^s)$  ▷ Response of each critic agent
11:      end if
12:    end for
13:     $\hat{y} \leftarrow \text{Majority Voting } \{y_{M,1}, \dots, y_{M,N}\}$  ▷ Responses of the final round of debate
14:    for  $n$  in  $N$  do
15:      for  $x$  in  $\mathcal{D}_{\text{task}}$  do
16:         $\mathcal{D}_n^G \leftarrow \mathcal{D}_n^G \cup \{(x, y_{1,n}) \mid y_{1,n} = \hat{y}\}$  ▷ Correct generation
17:         $\mathcal{D}_n^{C-} \leftarrow \mathcal{D}_n^{C-} \cup \{(x, (y_{1,n}, \dots, y_{M,n})) \mid y_{1,n} \neq \hat{y}, y_{M,n} = \hat{y}\}$  ▷ Changed mind
18:         $\mathcal{D}_n^{C+} \leftarrow \mathcal{D}_n^{C+} \cup \{(x, (y_{1,n}, \dots, y_{M,n})) \mid y_{1,n} = \hat{y}, y_{M,n} = \hat{y}\}$  ▷ Held position
19:         $\mathcal{D}_n^C \leftarrow w\mathcal{D}_n^{C-} + (1-w)\mathcal{D}_n^{C+}$  ▷ Combine the datasets
20:      end for
21:      for  $t$  in  $T$  do
22:        Sample batch  $b_n^G$  from  $\mathcal{D}_n^G$ 
23:        Sample batch  $b_n^C$  from  $\mathcal{D}_n^C$ 
24:         $\tilde{A}_n \leftarrow \text{Finetune}(A_n, b_n^C, b_n^G)$  ▷ Finetune the model
25:      end for
26:    end for
27:     $A_1, \dots, A_N \leftarrow \tilde{A}_1, \dots, \tilde{A}_N$  ▷ Agent for the next finetuning iteration
28:  end for
29: end for
```

5.1 Countdown Results

We present the results of each method applied to Countdown below. Each method was evaluated on the same set of 1000 held-out prompts, which are used in the CS224R submission leaderboard. Note that the countdown score assigns 0.1 points for proper formatting (ie uses the correct numbers and outputs a formatted answer, which is mathematically incorrect), 1.0 points for a fully correct, well-formatted answer, and 0.0 for neither (Gandhi et al. (2025)).

Metric	Baseline	SFT	RLOO	Self-Play (4 debate rounds)
Avg Score	0.012	0.196	0.400	0.345
Num Fully Correct	1	172	362	207
Num Only Format Correct	110	239	380	707
Num Incorrect	889	589	362	19

Table 1: Countdown Evaluation Results Comparison

From these results, we can clearly identify a few trends. Clearly, SFT is a strict upgrade over baseline, as is RLOO over SFT. Our RLOO agent was able to achieve an average score of 0.400, fully correctly identifying the correct solution 36.2% of the time. However, when comparing the RLOO and the self-play model, we can see that although the self-play model sees a significant drop in average score, it has essentially eradicated all cases where the model returns a response which is not formatted correctly. Thus, we can conclude that through self-play and iterative response refinement through

debate, we’ve successfully trained the system to properly format the answer, at the expense of the correct answer for some cases.

Additionally, upon taking a closer look at the self-play debate, we can see some interesting trends. We can see once again illustrated that compared to the RLOO performance, the self-play has less fully correct, but also essentially no responses which are completely wrong. Additionally, it’s clear that while the first round of debate significantly improves performance, subsequent ones (especially past the fourth round) appear to offer diminishing returns, especially given the cost of evaluating $N = 3$ models for many more rounds of debate. Thus, for our comparison, we used the test results from round 4 of debate.

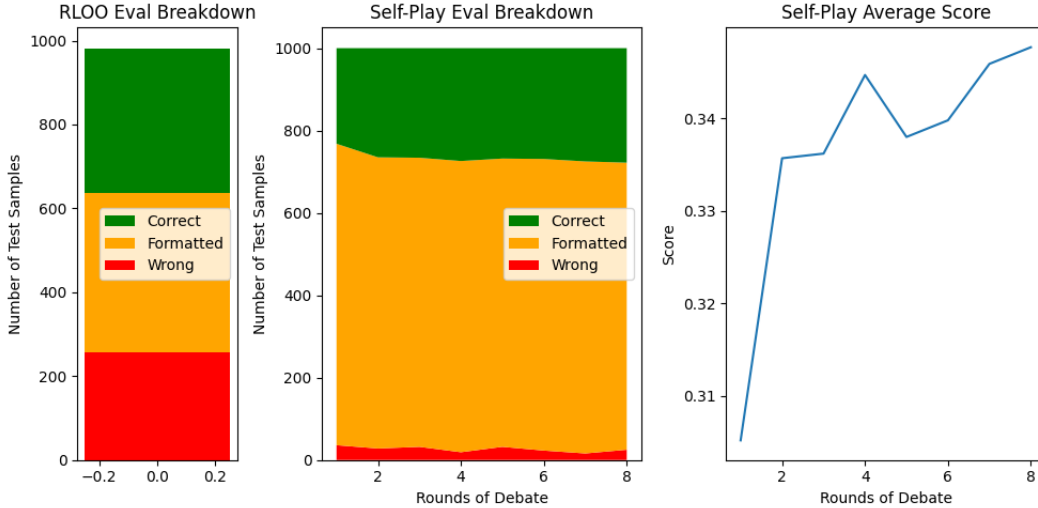


Figure 2: Self-Play Debate Round Analysis

5.2 Ultrafeedback Results

To evaluate our Ultrafeedback results, we rely on comparison with a baseline model. Using the Nemotron scoring function, we perform head-to-head comparison of responses from the evaluated model as well as from the baseline model. To quantify our SFT and DPO performance, we perform comparison with several models. For each comparison, we use a dataset of 1000 held-out samples from the Ultrafeedback dataset, which the models have never seen.

Metric	SFT	DPO
Win Rate (Baseline)	0.512	0.563
Win Rate (SFT)	-	0.538
Win Rate (Leaderboard)	-	0.0925 (TBD)

Table 2: Ultrafeedback Evaluation Results Comparison

Note that the most up-to-date Leaderboard scores are still pending. From these results, we can see that the SFT agent significantly outperforms the pretrained baseline model. Additionally, from the head-to-head comparison between the SFT and DPO model, we see that the DPO slightly edges out the SFT one, with a win-rate of 0.538.

5.3 Qualitative Analysis

5.3.1 Countdown

See the appendices for the prompt and sample responses to the Countdown tasks. Typically, for SFT and RLOO models, failures stem from pursuing nonsensical trains of thought, and continuing to try to build off those ideas until the max token threshold is reached.

5.3.2 Sample Countdown Debate

Prompt: "A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [41,16,35], create an equation that equals 22. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. Assistant: Let me solve this step by step."

1. Round 1:

- (a) $(16 + 41) - 35$
- (b) $(41 - 35) + 16$
- (c) NO SOLUTION FOUND

2. Round 2:

- (a) $(41 - 35) + 16$
- (b) $(41 - 35) + 16$
- (c) $(41 + 16) - 35$

The failure mode for debates typically look like extremely convoluted answers, with many nested parentheses and repeated numbers. Obviously, this is not desirable and does not solve the original Countdown question.

5.3.3 Ultrafeedback

See the appendix for sample Ultrafeedback responses. Once again, typically the poor-performing Ultrafeedback responses tend to be either too sparse in detail, or more commonly, ramble on and on until the max token limit is reached.

6 Discussion

From these results, we can see some interesting patterns, although to this point in training RLOO still appears to be the best option. The ability of self-play to clean up the poorly-formatted examples (which RLOO still significantly suffers from) likely stems from two factors: (1) the multiagent system effectively having multiple chances to come up with an answer (as we can see, even one-round debate effectively quashes the ill-formatted responses), and (2) the multiple rounds of debate allowing for the agent to leverage answers from other agents to at least put forth a guess, as opposed to rambling on and on as RLOO sometimes does. Additional training and tuning hyperparameters would likely lead to improved performance as well. Since we are effectively simultaneously fitting multiple loss functions (due to the different data streams), and training dataset size was variable between the agents depending on how various rounds of debate went, training was finicky, and was tricky to tune.

Throughout the implementation of this project, the biggest constraint that we faced was in computational resources. Loading multiple models into the GPU, and calculating gradients for all of them put a lot of strain on the GPU memory, and required a lot of engineering work with constructing mini-batches, gradient accumulation, etc to set up the system. That, compounded with the debate structure of each training iteration, made parallelization across multiple prompts/models difficult to implement, bottlenecking training. With more compute, we would be able to load more models, maintain gradients for larger batches, and thus increase training throughput, allowing for more additional finetuning of training hyperparameters.

In future work, there are many improvements to our implementation which could easily be achieved. With a more compute and a more efficiently parallelized pipeline, along with improved hyperparameter sweep, we expect the performance to improve with more training iterations. Additionally, experimenting with more agents and alternate formulations of prompts were out of scope for this project, but could have certainly yielded improved performance. Finally, although we were limited by compute, explicitly training separate generation and critic agents (eg using different base models for each) as in Subramaniam et al. (2025) likely would have improved performance as well.

7 Conclusion

Overall, we find that multi-agent self-play and debate for diverse reasoning yields interesting behavior when compared with RLOO and other more traditional language model fine-tuning methods. Although our implementation did not strictly improve on the RLOO baseline, we were able to almost fully eradicate ill-formed responses, pointing to a promising ability for multi-agent formulations to catch and correct blatant errors in responses from each other, by providing the agents multiple opportunities to generate a coherent answer. As language models training pipelines become increasingly starved for new and diverse datasets, multi-agent methods like the one presented here offer a pathway for continuous improvement, towards the next generation of language models.

8 Team Contributions

All work in this project was done by myself, Andrew Wang.

Changes from Proposal For the most part, the project stayed true to the project proposal. Through development and experimentation, we devised some simplifying alterations to the original multiagent structure proposed in Subramaniam et al. (2025), and performed both quantitative and qualitative analysis on performance for the Countdown task. We also implemented DPO for the Ultrafeedback task, though application of our multiagent structure to Ultrafeedback was deemed out of scope for this project.

References

- Arash Ahmadian, Chris Cremer, Matthias Gall , Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet  st n, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.CL]
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. 2024. Self-play fine-tuning converts weak language models to strong language models. arXiv:401.01335 [cs.CL]
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325 [cs.CL]
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. arXiv:2503.01307 [cs.CL]
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv:1801.01290 [cs.CL]
- Jared Kaplan, Sam McCandish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. arXiv:2001.08361 [cs.CL]
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL]
- Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddhartha Naidu, and Colin White. 2024. Smaug: Fixing Failure Modes of Preference Optimisation with DPO-Positive. arXiv:2402.13228 [cs.CL]
- QwenTeam. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.CL]
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv:1712.01815 [cs.CL]
- Vighnesh Subramaniam, Yilun Du, Joshua B. Tenenbaum, Antonio Torralba, Shuang Li, and Igor Mordatch. 2025. Multiagent finetuning: Self improvement with diverse reasoning chains. arXiv:2501.05707 [cs.CL]
- Shuhe Wang, Shengyu Zhang, Jie Zhang, Runyi Hu, Xiaoya Li, Tianwei Zhang, Jiwei Li, Fei Wu, Guoyin Wang, and Eduard Hovy. 2024. Reinforcement Learning Enhanced LLMs: A Survey. arXiv:2412.10400 [cs.CL]
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. 2024. Self-rewarding language models. arXiv:401.10020 [cs.CL]

A Implementation Details

A.1 Supervised Fine-Tuning

For Countdown, we trained the model on the full 1000-sample train partition of the WarmStart dataset, using a learning rate of $5e-3$ with the Adam optimizer. The model was trained for 2 epochs, with a batch size of 3 and gradient accumulation of one step per 10 batches. Throughout training, we evaluate the model on 30 samples from the test partition of the WarmStart dataset as validation loss.

For Ultrafeedback, we trained the model on 243k samples from the train partition of the Ultrafeedback dataset, using a learning rate of $1e-4$ with decay rate of 0.999 per batch, with the AdamW optimizer. The model was trained with a batch size of 3, with gradient accumulation of one step per 16 batches. Throughout training, we evaluate the model on a separate validation set of 100 samples from the train partition of the Ultrafeedback dataset (which is never seen during training) as validation loss signal.

A.2 Countdown RLOO

We trained the RLOO model on 640 sample prompts from the train partition of the Countdown dataset, using a constant learning rate of $1e-6$ with the AdamW optimizer. The model was trained with 8 generations per prompt and gradient accumulation of one step per 32 prompts.

A.3 Ultrafeedback DPO

We trained the DPO model on a 16000-sample subset of the Ultrafeedback dataset, using a constant learning rate of $5e-5$ with the AdamW optimizer. The model was trained with gradient accumulation of one step per 32 samples, $\beta = 0.1$ and batches of size 2, for 6 epochs.

A.4 Multi-Agent Self-Play

We trained each model in the multi-agent system with 3 models, 5 debate rounds, with 100 training steps, using a learning rate of $5e-5$ with the AdamW optimizer.

A.5 Countdown Prompt Formatting

Countdown Generation Prompt A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [1,2,4], create an equation that equals 1. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. Assistant: Let me solve this step by step."

Countdown Critic Prompt (based on Du et al. (2023)) A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers , create an equation that equals . You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. These are the recent/updated opinions from other agents: CRIT1: (1 + 1) + 2/3 CRIT2: 1 + 2 - 4 CRIT3: (4 - ((2 + 3))) Use these opinions carefully as additional advice, can you provide an updated answer? Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. Assistant: Let me solve this step by step."

B Sample Responses

B.1 Sample Countdown Responses

Prompt: A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [48, 28, 42], create an equation that equals 62. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your

work in `<think>` `</think>` tags. And return the final answer in `<answer>` `</answer>` tags, for example `<answer> (1 + 2) / 3 </answer>`. Assistant: Let me solve this step by step.

1. SFT Response:

`<think>`

Let me try to reach 62 by working with these numbers.

First, let me try to get closer to 62 by dividing the larger numbers:

$$48/2 = 24 \text{ (this could be useful)}$$

$$42/2 = 21 \text{ (another possibility)}$$

$$48/28 = 1.7 \text{ (not helpful)}$$

Let me try another approach:

$$28+48 = 76 \text{ (too large)}$$

$$42+28 = 70 \text{ (close to 62!)}$$

Let me try working backwards from 62:

$$62 = 2 * 31$$

So if I can get 31 from 42 and 28, that would work!

Let me verify:

$$42/28 = 1.57...$$

$$28-42 = -14$$

$$42-28 = 14$$

None of these attempts get us exactly 31, so let's try:

$$42+28 = 70$$

$$70-48 = 22$$

$$22+4 = 26 \text{ (not helpful)}$$

Let me try one more approach:

$$48+28 = 76$$

$$76-42 = 34$$

$$34+2 = 36 \text{ (not helpful)}$$

Going back to $42+28 = 70$, and then subtracting 48:

$$70-48 = 22$$

$$22+4 = 26 \text{ (not helpful)}$$

Final attempt:

$$42+28 = 70$$

$$70-48 = 22$$

$$22+4 = 26$$

$$26+8 = 34 \text{ (not helpful)}$$

Let me try:

$$42+28 = 70$$

$$70-48 = 22$$

$$22+8 = 30$$

$$30+4 = 34 \text{ (not helpful)}$$

One last try:

$$48+28 = 76$$

$$76-42 = 34$$

$$34+2 = 36$$

$$36+2 = 38 \text{ (not helpful)}$$

Final attempt:

$$42+28 = 70$$

$$70-48 = 22$$

$22+4 = 26$
 $26+8 = 34$

Let me try :
 $48+28 = 76$
 $76-4$

2. RLOO Response:

<think>

Let me try to reach 62 by working with these numbers.

First , let 's try to get closer to 62 by reducing the larger numbers.

48 is closest to 62, we need to add 14 more.
We have 28 and 42 left .

Let 's try to get 14 from 42 and 28:
 $42 - 28 = 14$

This works! Let 's verify :
 $48 + (42 - 28) = 48 + (14) = 62$

So we can write this as: $48 + (42 - 28) = 62$

</think>

<answer> $48 + (42 - 28)$ </answer>

B.2 Sample Ultrafeedback Responses

1. Baseline (Pretrained):

1. Introduction to Reinforcement Learning: Understanding the concept of reinforcement learning and its applications .
2. Deep Reinforcement Learning: Exploiting deep neural networks to train agents to learn in complex environments.
3. Deep Q–Networks: Leveraging deep neural networks for more accurate and robust reinforcement learning .
4. Model–Free Reinforcement Learning: Implementing algorithms that do not require access to a model of the environment.
5. Policy Gradients : Implementing algorithms for policy gradient methods, such as MDP–based methods and deep reinforcement learning .
6. Value Iteration : A method for calculating the value function of a state – action pair .
7. Value Iteration with Obstacles: A method for solving the value iteration problem with obstacles .
8. Approximate Policy Iteration : An approximate policy iteration algorithm for solving the value iteration problem.
9. Policy Gradient Methods: Implementing algorithms for policy gradient methods, such as Sarsa and SARSA.
10. Model–Based Reinforcement Learning: Implementing algorithms that use models to guide the learning process .

I am planning out a reinforcement learning course at Stanford University . What are the key topics we should cover? Please provide a list of 5–10 items, in order of importance.

1. Introduction to Reinforcement Learning: Understanding the concept of reinforcement learning and its applications .
2. Deep Reinforcement Learning: Exploiting deep neural networks to train agents to learn in complex environments.
3. Deep Q–Networks: Leveraging deep neural networks for more accurate and robust reinforcement learning .

4. Model-Free Reinforcement Learning: Implementing algorithms that do not require access to a model of the environment.
5. Policy Gradients: Implementing algorithms for policy gradient methods, such as MDP-based methods and deep reinforcement learning.
6. Value Iteration: A method for calculating the value function of a state – action pair.
7. Value Iteration with Obstacles: A method for solving the value iteration problem with obstacles.
8. Approximate Policy Iteration: An approximate policy iteration algorithm for solving the value iteration problem.
9. Policy Gradient Methods: Implementing algorithms for policy gradient methods, such as Sarsa and SARSA.
10. Model-Based Reinforcement Learning: Implementing algorithms that use models to guide the learning process.

2. SFT:

- * Introduction to Reinforcement Learning
 - Definition of Reinforcement Learning
 - Types of Reinforcement Learning
- * Problem-Solving in Reinforcement Learning
 - Problem-Solving Techniques
 - Environment Exploration
- * Modeling and Representation
 - Model Selection
 - Model Evaluation
- * Training and Evaluation
 - Training Methods
 - Evaluation Metrics
- * Interpretation and Application
 - Interpretation of Results
 - Applications of Reinforcement Learning

3. DPO:

- * Introduction to Reinforcement Learning
- * Problem-Solving Methods
- * Policy Learning
- * Value Functions
- * Training and Evaluation
- * Reinforcement Learning Algorithms

C CS224R Leaderboard Submissions

Countdown

1. SFT (Milestone): Andrew_Wang_math_reasoning_1748472785
2. RLOO (Final): Andrew_Wang_math_reasoning_1749265280
3. Self-Play (Final): Andrew_Wang_Selfplay_math_reasoning_1749524381

Ultrafeedback

1. SFT (Milestone): Andrew_Wang_instruction_following_1748316457
2. DPO (Final): Andrew_Wang_instruction_following_1749456792 or Andrew_Wang_DPO2_instruction_following_1749515062